

Reasoning with Language Model is Planning with World Model



Agentic AI: 3. Tutorial

Cono Cirone and Esther Giuliano

25. August 2026

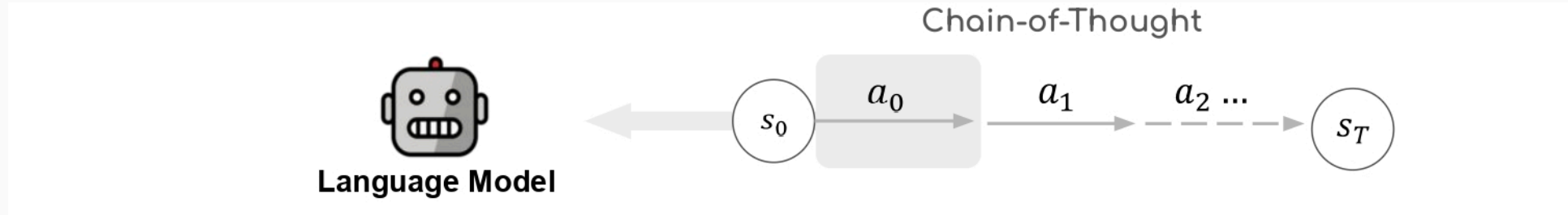
SoSe 2026, Bielefeld University



RAP: Reasoning via Planning	2
RAP: Reasoning via Planning	5
RAP: Reasoning via Planning	6
Our Implementation: Overview	7
world_model.py — Predicting the Next State	8
Probability Approximation	9
mcts_node.py — Reward and Node Expansion	11
mcts.py — UCT Selection	12
search.py — Putting It All Together	13
Plan Evaluation	14
Blocksworld Dataset	15
Methods Compared	16
Results	17
Bibliography	18
About	19



Recent LLMs approaches:



These approaches struggle with:

- planning, long-horizon tasks and multi step reasoning
- generating correct action sequences

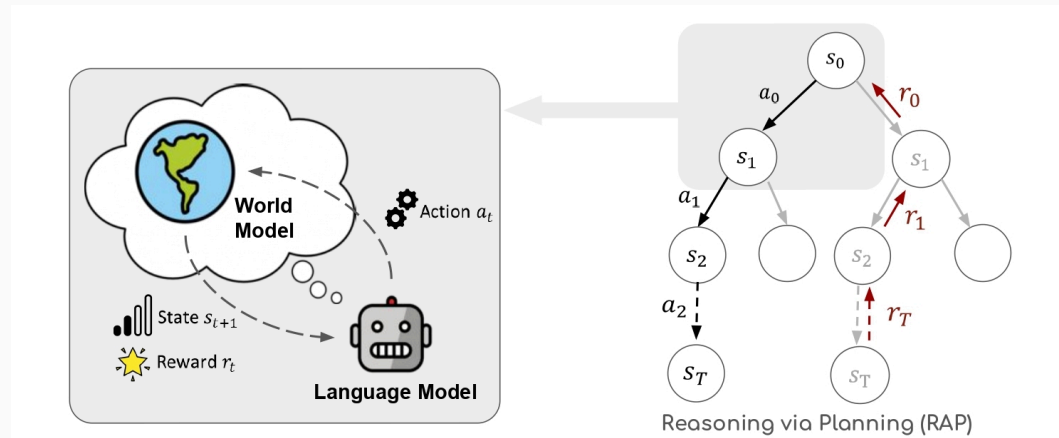
Reasons:

- answer generated left-to-right in one shot
- no ability to look ahead to see if a sequence of actions will actually reach the goal or end up in a dead end
- no ability to explore multiple possible plans and compare them

Hao et al., 2023



- 💡 Turning the reasoning problem into a search tree



- The LLM has two jobs: **reasoner** (proposes next action) and **world model** (predicts what will happen after the action)
- RAP builds a search tree to select the next action where where:
 - Each **node** represents a specific **state** in the planning process, and contains a **reward** value that measures how promising the plan is
 - Each **edge** represents the taken **action**



- Best action selection with the **Monte Carlo Tree Search** algorithm in multiple rollouts:

```
1  1. SELECT    – Walk down the tree from the root, choosing the
                        most promising branch at each step using the UCT formula.
2
3  2. EXPAND    – At the selected node, ask the LLM:
4                      (a) "What does the world look like after this action?" → r1
5                      (b) "Is this action a step toward the goal?" → r0
6                      Create one child node per valid action.
7
8  3. BACKPROPAGATE – Walk back up the path just taken, updating the
9                      running statistics (visit count, total reward) at
10                     every node we passed through.
```

pseudocode



- **UCT** formula

$$\text{UCT}(v) = \underbrace{M(v)}_{\text{best reward seen}} + w_{\text{exp}} c \cdot \underbrace{\sqrt{\frac{\ln N(\text{parent})}{N(v)}}}_{\text{exploration bonus}} \quad (1)$$

where:

- $M(v)$ = best cumulative reward seen through node v so far
- $N(v)$ = how many times node v has been visited
- $w_{\{\text{exp}\}}$ = exploration weight (set to 1.0)



- The **reward** formula is: $r(s, a) = r_0(s, a)^\alpha * r_1(s, a)^{1-\alpha}$
where
 - r_0 is the **action score**, i.e. how promising is this specific action from this state?
 - r_1 is the **state score**, how close is the resulting state to the goal?
 - α the **balance weight** = 0.5
- After all rollouts are done, the path with the **highest average reward** is chosen.



exercise_3/

world_model.py The two LLM calls RAP needs (predict state + score action)

mcts_node.py One node in the search tree, with its expansion logic

mcts.py The MCTS algorithm (select → expand → backpropagate)

search.py Ties everything together for one Blocksworld problem

Supporting Files:

utils.py Data loading: PDDL parser + deterministic plan simulator

baseline.py Direct LLM planner (no search) for comparison

run.py Full experiment: loads data, runs both methods, prints results



The LLM is called with a **different few-shot prompt** depending on the action type:

```
1  async def predict_next_state(self, last_state: str, action: str) -> str:
2      if "Pick" in action:
3          prompt_template = self.prompts["world_update_pickup"]
4      elif "Unstack" in action:
5          prompt_template = self.prompts["world_update_unstack"]
6      elif "Put" in action:
7          prompt_template = self.prompts["world_update_putdown"]
8      else:
9          prompt_template = self.prompts["world_update_stack"]
10
11     filled_prompt = prompt_template.format(last_state, action)
12     change_text = await self._call(system, filled_prompt, max_tokens=150)
13     return change_text # e.g. "[CHANGE] the red block is now in the hand ..."
```

The [CHANGE] text is then passed to `apply_change()` (from the original RAP repo) to produce the new state string. The `score_action` call is discussed next.



```
1  async def score_action(self, state0: str, state1: str, goal: str) -> float:
2      filled_prompt = self.prompts["confidence"].format(state0, state1, goal)
3      system = (
4          "You are evaluating a block-stacking plan. "
5          "Answer the final question with only 'Yes' or 'No'."
6      )
7      answer = await self._call(system, filled_prompt, max_tokens=10)
8      if answer.lower().startswith("yes"):
9          return 0.9
10     else:
11         return 0.1
```

py



The Core Limitation: The original paper relies on **implicit signals** via raw token logits, which are unavailable over standard chat APIs.

▶ **Original Paper (LLaMA)**

- ▶ **Mechanism:** Token log-likelihoods ($\log P(\text{action} \mid \text{context})$).
- ▶ **Intuition:** “How naturally would the LLM generate this action next?”
- ▶ **Dependency:** Requires running models locally to access logits.

▶ **Our Implementation**

- ▶ **Mechanism:** Explicit Yes/No prompt converted to a float (Yes $\rightarrow$ 0.9, No $\rightarrow$ 0.1).
- ▶ **Intuition:** “Is this state closer to the goal than before?”
- ▶ **Dependency:** Works on **any** standard chat API (via AgentScope).



The **reward** property directly implements the paper formula:

```
1 @property
2 def reward(self) -> float:
3     if self._r0 < 0 or self._r1 < 0:
4         return min(self._r0, self._r1)      # propagate failure
5     return self._r0 ** self._r_alpha * self._r1 ** (1 - self._r_alpha)
```

r_1 is computed by counting satisfied goal conditions after the LLM predicts the new state:

```
1 goals      = re.findall(r"the \w+ block is on top of the \w+ block", self.goal)
2 meetings   = [g in new_state for g in goals]
3
4 if all(meetings):
5     self._r1 = 100.0                        # all goals met → terminal success
6 else:
7     self._r1 = sum(meetings) / len(meetings) + 0.5  # partial credit
```



Each rollout is just two steps:

```
1 async def rollout(self, root):  
2     path = await self._select_prior(root)    # walk down, expanding as we go  
3     self._back_propagate(path)              # update Q, N, M on the way back
```

py

The child picked at each node maximises the **UCT score**:

```
1 def _uct_score(self, node, log_n_parent):  
2     if self.N[node] == 0:                    # unvisited: prioritise exploration  
3         return node.reward + self.w_exp * math.sqrt(log_n_parent)  
4  
5     exploitation = self.M[node]              # best reward seen through this node  
6     exploration  = self.w_exp * math.sqrt(log_n_parent / self.N[node])  
7     return exploitation + exploration
```

py

$M[\text{node}]$ (best seen) is used instead of $Q[\text{node}]/N[\text{node}]$ (mean) to avoid penalising paths that found a good plan early.



Root node is bootstrapped from the problem text, then MCTS runs for rollouts iterations:

```
1  root_prompt = (  
2      prompts["goal_prefix"] + goal + "\n"  
3      + prompts["state_prefix"].format(0) + " " + initial_state + "\n"  
4  )  
5  root = BlocksworldMCTSNode(prompt=root_prompt, depth=0, r0=0.0, ...)  
6  mcts = MCTS(w_exp=1.0, aggr_reward="mean", aggr_child="max")  
7  
8  for i in range(rollouts):  
9      await mcts.rollout(root)  
10     best_node, best_reward = mcts.best_terminal(root)  
11     trajectories.append(best_node.prompt)
```

After all rollouts, `best_terminal(root)` walks the tree and returns the terminal node with the highest mean path reward. `extract_plan(best_node.prompt)` then reads all [ACTION N] lines to produce the final action sequence.



A plan is **successful** if and only if *both* hold when simulated by our deterministic state machine:

1. **Every action is legally applicable** — no precondition is violated at any step
e.g. you cannot pick up a block if your hand is already holding one
2. **The final state satisfies all goal conditions** — every stacking condition in the goal appears in the final state



Split	Instances	GT plan length	Difficulty
step_2	30	2 actions	Easy
step_4	57	4 actions	Medium
step_6	114	6 actions	Hard

Each instance is a .pddl file with 4 blocks (red, blue, orange, yellow). Each entry contains the PDDL file path, the ground-truth plan, and the step count.



Method	How it works
Baseline	One-shot: show problem + 4 few-shot examples → ask for a plan directly
RAP-MCTS	10 rollouts $\times$ (state prediction + action scoring per valid action)

Note

The wider the gap between baseline and RAP, the more the search is adding value.
Expected to grow with plan length.



	Our Baseline (Qwen3-VL-4B)	Our RAP-MCTS (Qwen3-VL-4B)	Paper RAP-MCTS (LLaMA-33B)
step_2 (Easy, n=30)	66.7%	100% ↑33.3pp	~100%
step_4 (Medium, n=57)	28.1%	63.2% ↑35.1pp	~86%
step_6 (Hard, n=114)	1.7%	5.2% ↑3.5pp	~42%



Hao, S., Gu, Y., Ma, H., Hong, J. J., Wang, Z., Wang, D. Z., & Hu, Z. (2023). *Reasoning with Language Model is Planning with World Model*. 20. <https://doi.org/10.48550/arXiv.2305.14992>



Original slide author: **Ester Giuliano, Cono Cirone**

Originally created for the seminar

Agentic AI: 3. Tutorial

at Bielefeld University SoSe 2026

